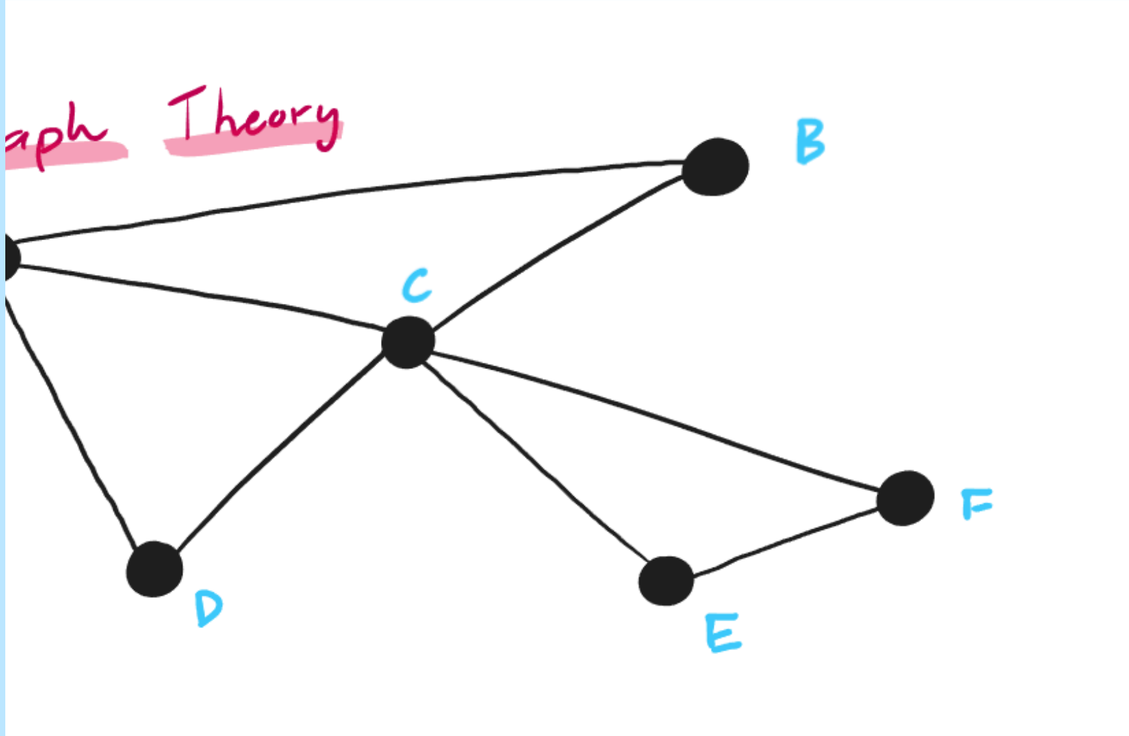
Graph Theory

SHSID CPC

Vocabulary

- Vertices (Nodes): ‘Dots’ on the graph
- Edges: ‘Lines’ between the dots
- Undirected Graphs: In the graph, we have edges that are not directed, in other words, both vertices they connect can traverse to one another
- Directed Graphs: In the graph, we have edges that are directed. This means we can travel in a one-sided fashion only from one node to another, and not the other way around



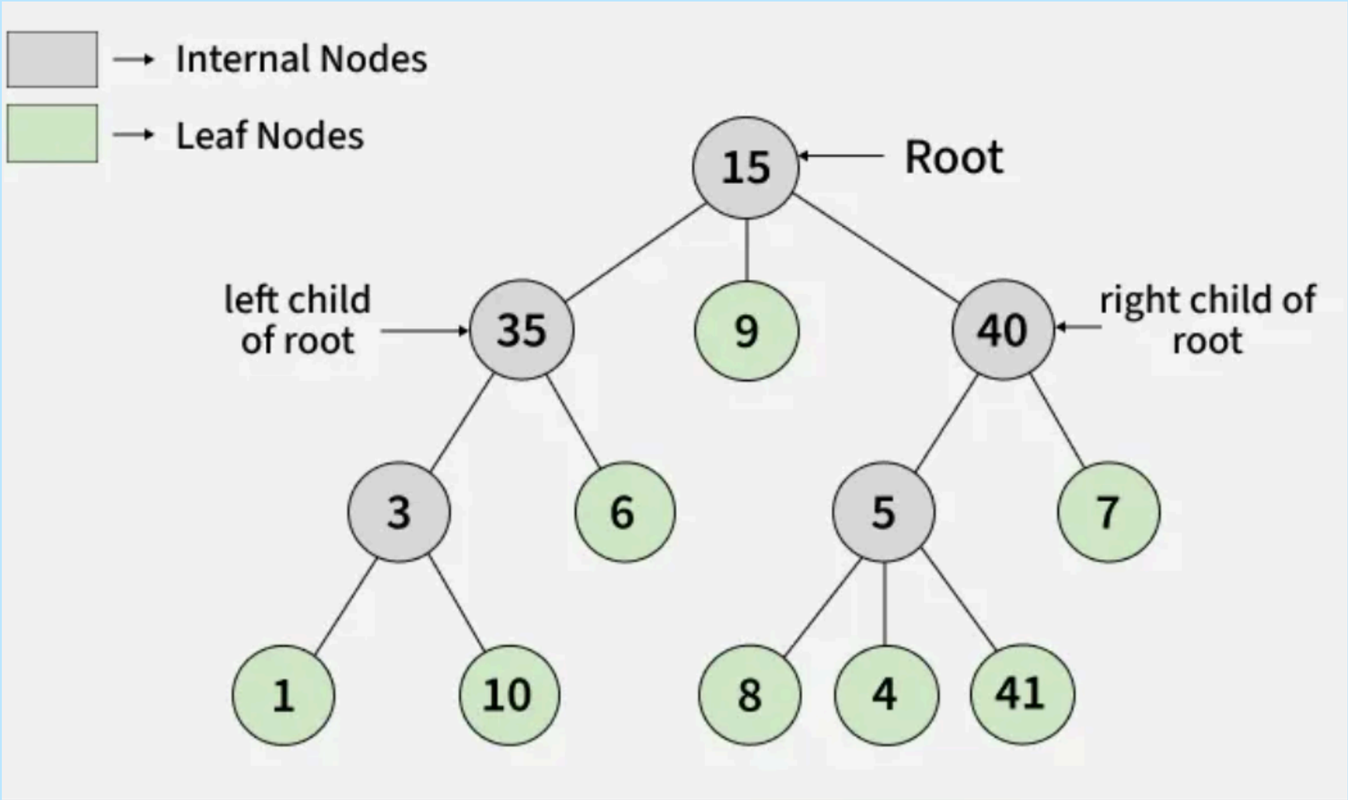
- (Note that we can express the undirected graph in a way of directed graphs, by which we remove every undirected edge and add two directed edges that connect both nodes)

Vocabulary (Continued)

- Unweighted Graphs: Graphs that have no weights on their edges
- Weighted Graphs: Graphs that, as the name suggests, have weights on their edges
- Cyclic: A graph that consists of a single cycle
- Acyclic: A graph that doesn’t have a cycle
- Tree: An undirected graph with no cycles
- Heap: A special type of trees

Trees

- Parent Node: The immediate predecessor of a node
- Child Node: The immediate successor of a node
- Root Node: A node that doesn’t have a parent node (usually drawn at topmost of a tree graph)
- Leaf Node: A node that doesn’t have any child nodes
- Ancestor: Ancestor of node X is a node on the unique path from the root of the tree to node X
- Descendant: The descendant of node X is any node that has node X as its ancestor
- Sibling: Nodes that have the same parent node



Trees (Continued)

- Level: The number of edges that are in the unique path from a node to the root node (or the number of ancestors that a node has)
- Internal Node: A node that has at least one child node
- Neighbor Node: The parent node or child node of a node
- Sub-tree: Consisting of a node and all its descendants
- Binary Tree: Each node has at most two child nodes

Representations

Adjacency matrix:

$$\text{adj}[u][v] = \begin{cases} 1 \text{ if there is an edge between u and v} \\ 0 \text{ otherwise} \end{cases}$$

Pros: Instant check for existing edge

Cons: Takes $O(V^2)$ Memory

Adjacency List

$$\text{adj}[v_i] = [v_1, v_2, \ldots]$$

adj[u] stores a list of all nodes that u is directly connected to

Pro: memory efficient O(V+E)

Con: slower to check for edge existence

Representations

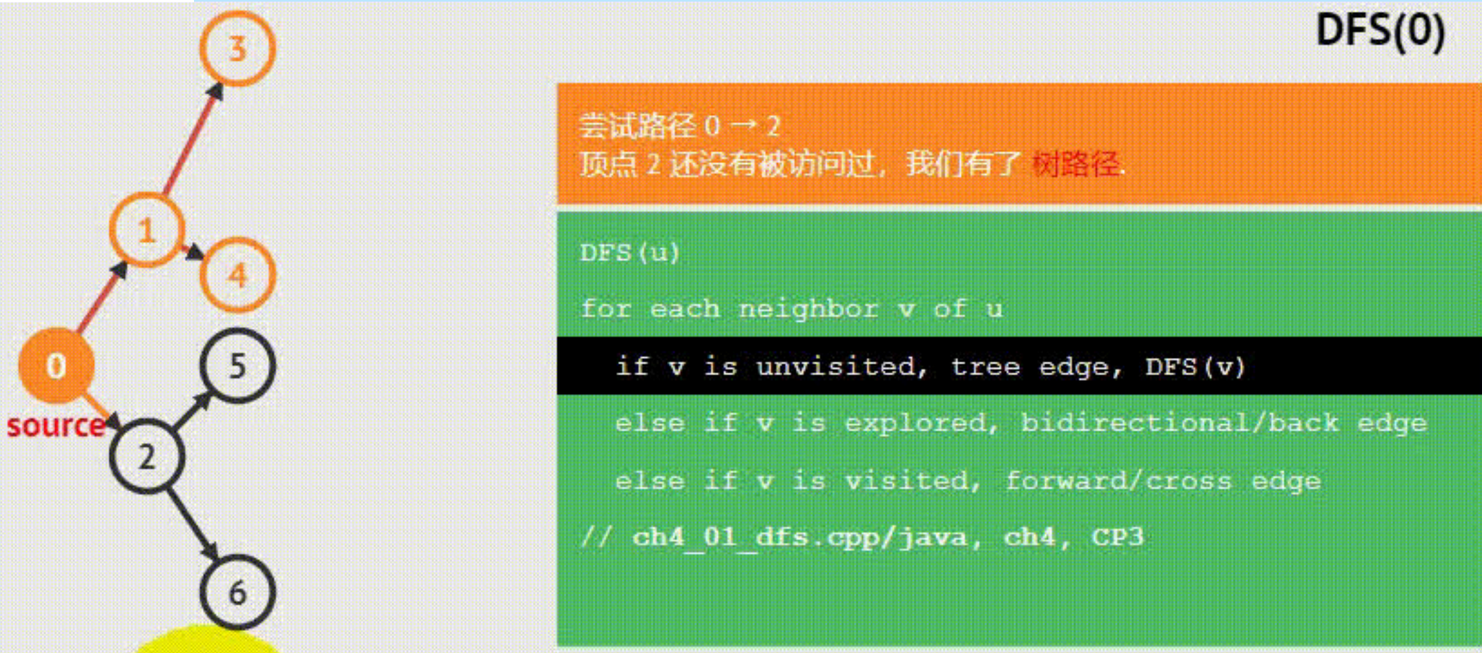
```python
def init_adj_matrix(V,E,directed = False):
    adj_matrix = [[0 for _ in range(V)] for _ in range(V)]
    for i in range(len(E)):
        # E is a list of tuples (vi, vj)
        adj_matrix[E[i][0]][E[i][1]] = 1
        if not directed:
            adj_matrix[E[i][1]][E[i][0]] = 1
    return adj_matrix
```

```python
def init_adj_list(V,E,directed=False):
    adj_list = [[] for _ in range(V)]
    for i in range(len(E)):
        adj_list[E[i][0]].append(E[i][1])
        if not directed:
            adj_list[E[i][1]].append(E[i][0])
    return adj_list
```

Graph Traversal: DFS

```python
def dfs(graph, node, visited=None):
  if visited is None:
    visited = set()
  visited.add(node)
  print(node, end=' ')
  for neighbour in graph[node]:
    if neighbour not in visited:
      dfs(graph, neighbour, visited)
```

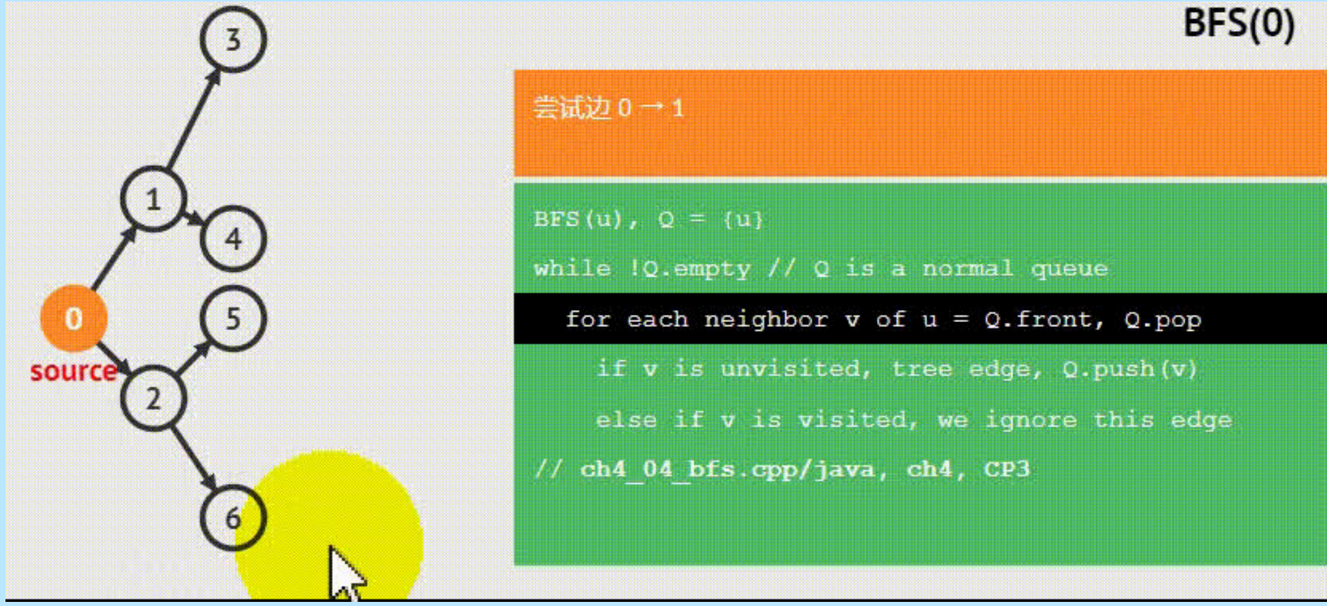
$$O(V + E)$$



Graph Traversal: BFS

```python
from collections import deque

def bfs_directed_graph(n, edges, start):
    """
    有向图 BFS 标准模板
    :param n: 图的节点个数（节点编号 0~n-1）
    :param edges: 有向边列表，格式 [[u1, v1], [u2, v2], ...]  u->v
    :param start: 遍历起点
    :return: BFS 遍历顺序
    """
    # 1. 构建邻接表（有向图只存单向边）
    adj = [[] for _ in range(n)]
    for u, v in edges:
        adj[u].append(v)  # 仅添加 u -> v 一条边

    # 2. 初始化访问标记 + 结果列表
    visited = [False] * n
    res = []

    # 3. 初始化队列，起点入队并标记
    q = deque()
    q.append(start)
    visited[start] = True

    # 4. BFS 核心循环
    while q:
        # 取出队首节点
        cur = q.popleft()
        res.append(cur)
        # 遍历当前节点的所有邻接节点（有向出边）
        for neighbor in adj[cur]:
            if not visited[neighbor]:
                visited[neighbor] = True
                q.append(neighbor)
    return res
```

$$O(V+E)$$



Connected Components

- Connected components are a group of nodes that always has at least one path between one node and another
- We can determine if two nodes are within the same connected component via DFS or BFS
- We would go through all the nodes that can be searched from a node using DFS or BFS, and then label them as part of the connect component of the original node
- Nodes in different connected components do not have a path between them

Sample T1

题目背景

复制 Markdown　中文　展开　进入 IDE 模式

在峰会期间，武装部队得处于高度戒备。警察将监视每一条大街，军队将保卫建筑物，领空将布满了 F-2003 飞机。

此外，巡洋船只和舰队将被派去保护海岸线。不幸的是，因为种种原因，国防海军部仅有很少的几位军官能指挥大型海战。因此，他们培养了一些新海军指挥官。军官们选择了“海战”游戏来帮助他们学习。

题目描述

在一个方形的盘上，放置了固定数量和形状的船只，每只船却不能碰到其它的船。在本题中，我们认为船是方形的，所有的船只都是由图形组成的方形。

求出该棋盘上放置的船只的总数。

输入格式

第一行为两个整数 R 和 C，用空格隔开，分别表示游戏棋盘的行数和列数。

接下来 R 行，每行 C 个字符，为 `#` 或 `.`。`#` 表示船只的一部分，`.` 表示水。

输出格式

一行一个字符串，如果船的位置放得正确（即棋盘上只存在相互之间不能接触的方形，如果两个 `#` 号上下相邻或左右相邻却分属两艘不同的船只，则称这两艘船相互接触了）。就输出 `There are S ships.`，S 表示船只的数量。否则输出 `Bad placement.`。

输入输出样例

输入 #1　复制

```
6 8
.....#.#
##.....#
##.....#
.......#
#......#
#..#...#
```

输出 #1　复制

```
There are 5 ships.
```

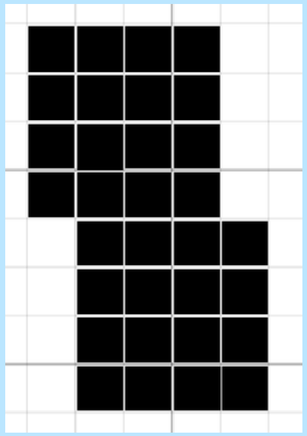
说明/提示

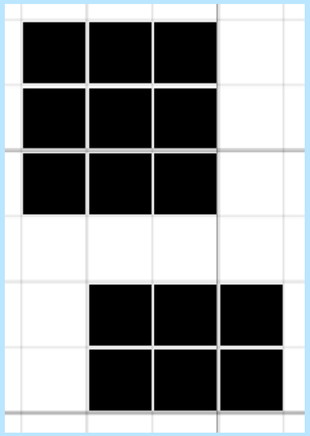
对于 100% 的数据，$1 \le R, C \le 1000$。

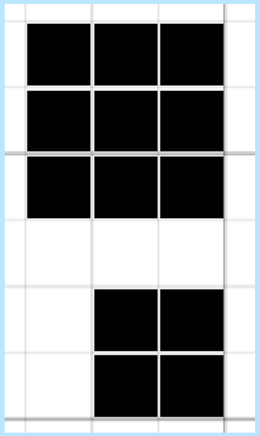
Solution

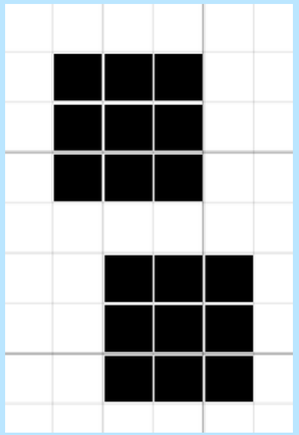
We realize that this is just simply a connected component problem. All we need to do is to find out (and label) all the connected components, and then count the number of connected components there are.

Note that we need to print out a “bad placement” whenever we have a connected component that isn’t in the shape of a square.









Sample T2

题目背景

复制 Markdown 中文 展开 进入 IDE 模式

高手最近谈恋爱了。不过是单相思。“即使是单相思，也是完整的爱情”，高手从未放弃对它的追求。今天，这个阳光明媚的早晨，太阳从西边缓缓升起。于是它找到高手，希望在晨读开始之前和高手一起在鳌头山上一起散步。高手当然不会放弃这次梦寐以求的机会，他已经准备好了一切。

题目描述

鳌头山上有 n 个观景点，观景点两两之间有游步道共 m 条。高手的那个它，不喜欢太刺激的过程，因此那些没有路的观景点高手是不会选择去的。另外，她也不喜欢去同一个观景点一次以上。而高手想让他们在一起的路程最长（观景时它不会理高手），已知高手的穿梭机可以让他们在任意一个观景点出发，也在任意一个观景点结束。

输入格式

第一行，两个用空格隔开的整数 n、m. 之后 m 行，为每条游步道的信息：两端观景点编号、长度。

输出格式

一个整数，表示他们最长相伴的路程。

输入输出样例

输入 #1 复制

```
4 6
1 2 10
2 3 20
3 4 30
4 1 40
1 3 50
2 4 60
```

输出 #1 复制

```
150
```

说明/提示

对于 100% 的数据：$n \le 20$，$m \le 50$，保证观景点两两之间不会有多条游步道连接。

Solution

Brute force DFS for every node

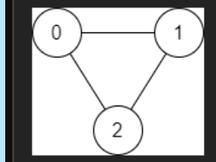
Sample T3

There is a **bi-directional** graph with `n` vertices, where each vertex is labeled from `0` to `n - 1` **(inclusive)**. The edges in the graph are represented as a 2D integer array `edges`, where each `edges[i] = [$u_i$, $v_i$]` denotes a bi-directional edge between vertex `$u_i$` and vertex `$v_i$`. Every vertex pair is connected by **at most one** edge, and no vertex has an edge to itself.

You want to determine if there is a **valid path** that exists from vertex `source` to vertex `destination`.

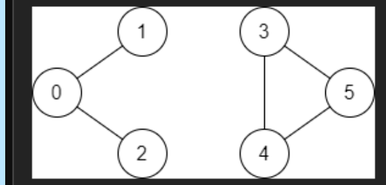
Given `edges` and the integers `n`, `source`, and `destination`, return `true` *if there is a **valid path** from* `source` *to* `destination`*, or* `false` *otherwise.*

Example 1:




```
Input: n = 3, edges = [[0,1],[1,2],[2,0]], source = 0, destination = 2
Output: true
Explanation: There are two paths from vertex 0 to vertex 2:
- 0 → 1 → 2
- 0 → 2
```

Example 2:




```
Input: n = 6, edges = [[0,1],[0,2],[3,5],[5,4],[4,3]], source = 0, destination = 5
Output: false
Explanation: There is no path from vertex 0 to vertex 5.
```

Constraints:

- `1 <= n <= 2 * 10^5`
- `0 <= edges.length <= 2 * 10^5`
- `edges[i].length == 2`
- `0 <= $u_i$, $v_i$ <= n - 1`
- `$u_i$ != $v_i$`
- `0 <= source, destination <= n - 1`
- There are no duplicate edges.
- There are no self edges.

Solution

DFS / BFS

Sample T4

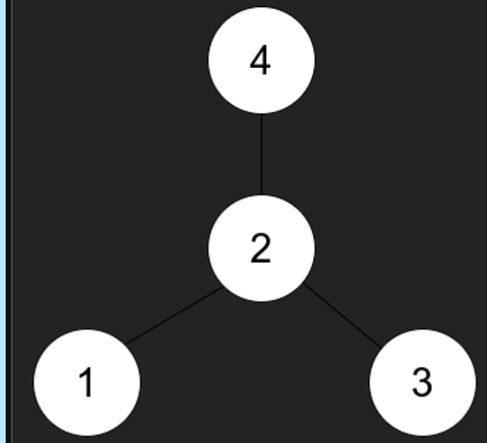
1791. Find Center of Star Graph

Easy | Topics | Companies | Hint

There is an undirected **star** graph consisting of `n` nodes labeled from `1` to `n`. A star graph is a graph where there is one **center** node and **exactly** `n - 1` edges that connect the center node with every other node.

You are given a 2D integer array `edges` where each `edges[i] = [$u_i$, $v_i$]` indicates that there is an edge between the nodes `$u_i$` and `$v_i$`. Return the center of the given star graph.

Example 1:




```
Input: edges = [[1,2],[2,3],[4,2]]
Output: 2
Explanation: As shown in the figure above, node 2 is connected to every other node, so 2 is the center.
```

Example 2:

```
Input: edges = [[1,2],[5,1],[1,3],[1,4]]
Output: 1
```

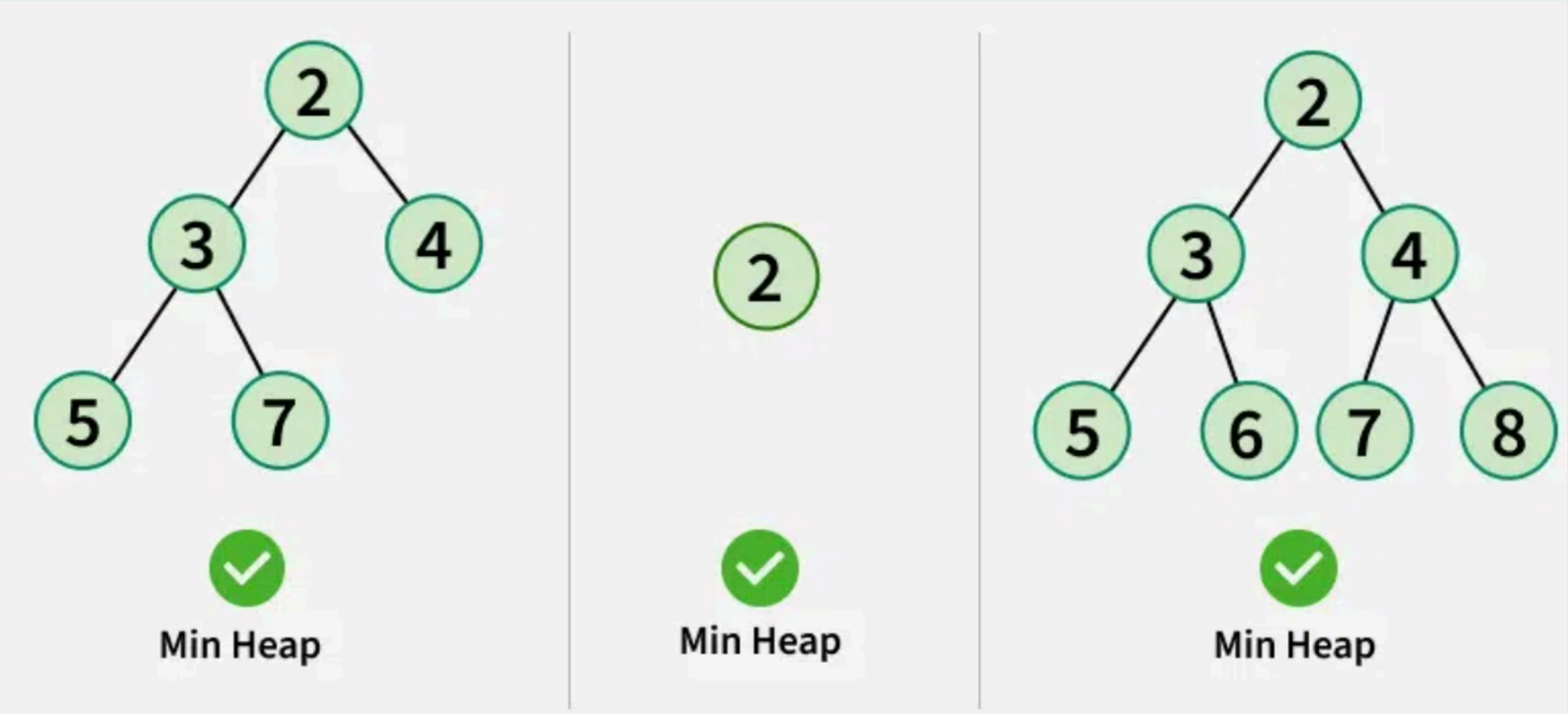
Constraints:

- `3 <= n <= $10^5$`
- `edges.length == n - 1`
- `edges[i].length == 2`
- `1 <= $u_i$, $v_i$ <= n`
- `$u_i$ != $v_i$`
- The given `edges` represent a valid star graph.

Solution

Count how many connections each vertex / node has

t4.py > find_center

```python
def find_center(edges):
    d = {}
    max = 0
    ind = 0
    for i in range(len(edges)):
        if edges[i][0] in d:
            d[edges[i][0]] = 1
        else:
            d[edges[i][0]] += 1
        if d[edges[i][0]] > max:
            max = d[edges[i][0]]
            ind = edges[i][0]
        if edges[i][1] in d:
            d[edges[i][1]] = 1
        else:
            d[edges[i][1]] += 1
        if d[edges[i][1]] > max:
            max = d[edges[i][1]]
            ind = edges[i][1]
    return ind
```

Heaps